

BETRIEBSSYSTEME

Technische Hochschule Mittelhessen

Andre Rein

– Betriebssysteme Scheduling –

SCHEDULING

DISCLAIMER:

Prozess = Thread = Task

GRUNDKONZEPT

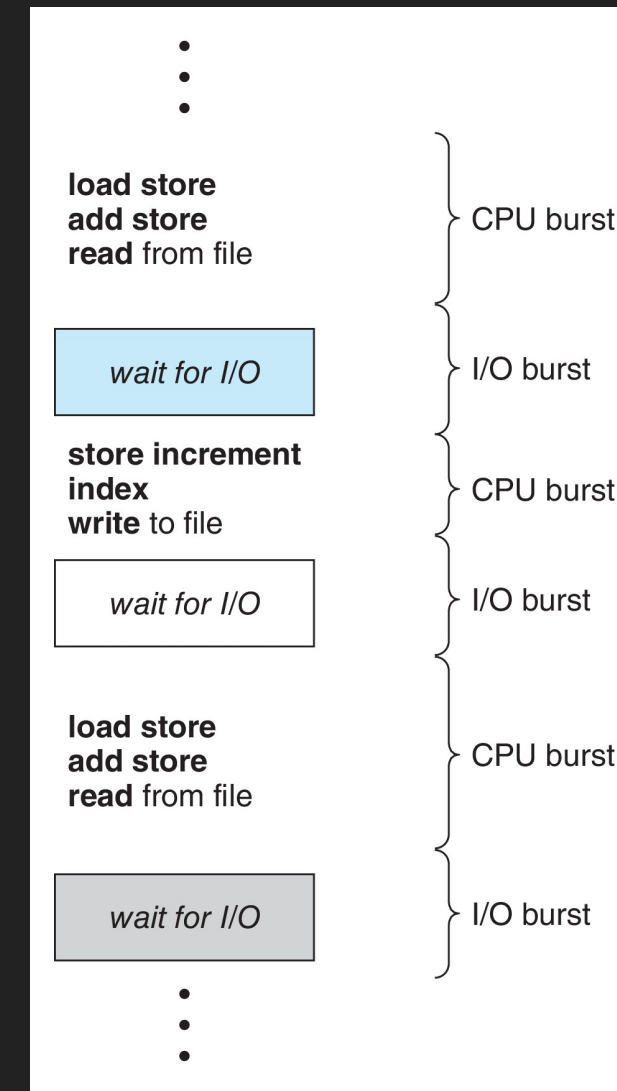
Auf einem einfachen System mit nur **einem** Core, kann nur ein einzelner Prozess zur gleichen Zeit ausgeführt werden

- Bei einem I/O Request hat die CPU nichts zu tun
 - Dieses **Nichtstun** ist schlicht verschwendete Zeit
- Bei Multiprogramming versucht man diese verschwendete Zeit für sinnvolle anderen Aufgaben zu nutzen
 - D.h. in der Wartezeit wird ein anderer Prozess auf der CPU ausgeführt
- Muss dieser Prozess wiederum warten (z.B. IO), wird die CPU einem dritten Prozess zugeteilt
 - Oder dem ersten Prozess, falls dieser seine I/O-Operation abgeschlossen hat

Die Grundidee ist also: Muss ein Prozess auf ein Ereignis warten, wird die CPU einem anderen Prozess zugeteilt.

GRUNDKONZEPT: CPU- UND IO-BURST

- Typisch für Programme ist ein ständiger Wechsel zwischen
 - CPU-Burst
 - IO-Burst
- Beispiel:
 - Lesen und Schreiben in einer Datei + Operationen
- CPU-lastige Prozesse → lange CPU Bursts, kurze IO-Bursts
- IO-lastige Prozesse → kurze CPU Bursts, lange IO-Bursts



CPU-SCHEDULING

Wenn die CPU bzw. ein CPU-Core ungenutzt ist, wählt der CPU-Scheduler einen Prozess aus der **ready**-Queue aus und weist den ungenutzten Core an diesen Prozess zu

Bei Multi-Core stehen einfach nur mehr Ressourcen (Cores) zur Verfügung

- Die Reihenfolge, also wann welcher Prozess einem Core zugewiesen wird, entscheidet der Scheduling-Algorithmus
- Die Datenstrukturen zur Verwaltung der **ready**-Queue können Verkettete Listen, Bäume oder andere Datenstrukturen sein

PREEMPTIVE UND NON-PREEMPTIVE

CPU-Scheduling Entscheidungen erfolgen bei folgenden Prozesszuständen

1. Wechsel von **running** zu **waiting**
2. Wechsel von **running** zu **ready**
3. Wechsel von **waiting** zu **ready**
4. Wechsel von **running** zu **terminated**

- Scheduling unter **1** und **4** ist **non-preemptive** (kooperierend) → Hat ein Prozess die CPU, gibt er diese **selbstständig ausschließlich** durch warten frei oder bei Beendigung
- Anmerkung: **2** und **3** kann nur dann auftreten, wenn das BS selbst CPU-Zeit erhält!

PREEMPTIVE UND NON-PREEMPTIVE

- Alle modernen Betriebssysteme verwenden **preemptive** Scheduler-Algorithmen
→ d.h. Prozesse wechseln **unfreiwillig** zwischen **running** und **ready** Queues
- Nur mit preemptivem Scheduling kann jeder Prozess regelmäßig Fortschritte machen (multi-tasking Anforderung) und
- asynchrone Ereignisse wie (Interrupts) vernünftig unterstützen
 - d.h. *jeder* Prozess muss *jederzeit* unterbrechbar sein

Frage: Welche Probleme können bei preemptive Scheduling auftreten?

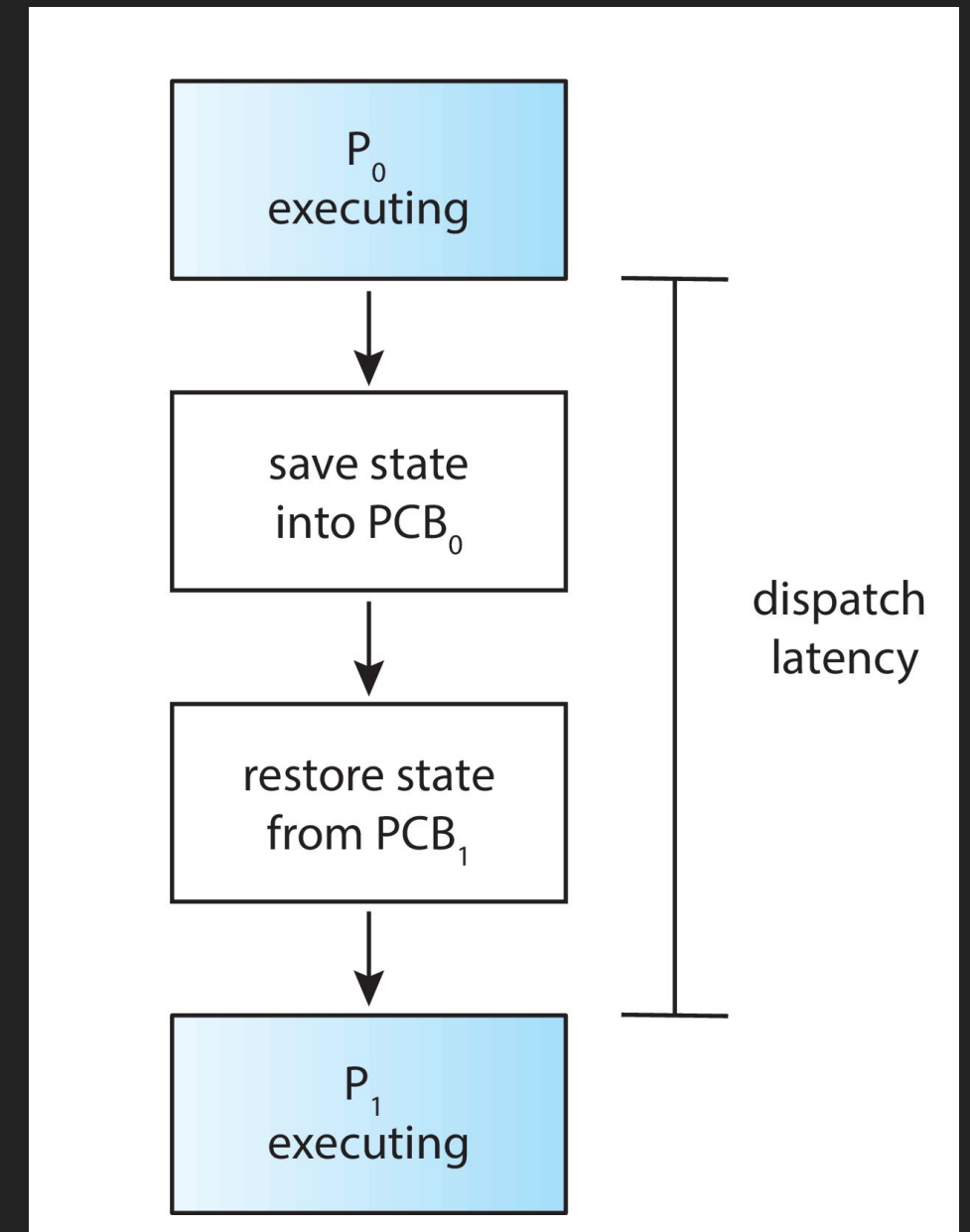
PREEMPTIVE UND NON-PREEMPTIVE

- Race Conditions bei geteilten Daten
- (**Deadlocks** bei Verwendung von Synchronisationsmethoden (Mutex, Semaphoren, Monitoren), zur Behebung der Race Conditions!)

Das größte Problem: **Inkonsistenz von Daten**. Wird z.B. der Kernel selbst bei einer "wichtigen" Aufgabe unterbrochen (z.B. Aktualisierung von Datenstrukturen im PCB oder System Call) und ein anderer Prozess arbeitet mit den falschen Daten weiter wird Chaos entstehen!

DISPATCHER

- Der Dispatcher veranlasst den Kontextwechsel von einem Prozess zu einem anderen Prozess
- Wechsel des Kontextes durch Retten des Ausführungszustandes und Austausch der PCBs
 - CPU-Register und andere Zustandsinformationen im PCB speichern
 - Neuen PCB laden, CPU-Register aktualisieren
- Wechsel in der User-Modus
- Anspringen der aktuellen Code-Stelle mittels Program Counter (PC)
- *Dispatch-Latenzzeit* (dispatch latency) → Zeit, die der Dispatcher benötigt, um einen Prozess zu stoppen und einen anderen zu starten



KRITERIEN FÜR SCHEDULING-ALGORITHMEN

- **CPU-Auslastung** → CPU Cores maximal ausgelastet halten
- **Durchsatz** → Anzahl der Prozesse, die ihre Ausführung pro Zeiteinheit abschließen, maximieren
- **Durchlaufzeit** → Zeitaufwand für die Ausführung eines **bestimmten** Prozesses minimieren
- **Wartezeit** → Zeitspanne, in der ein Prozess in der **ready**-Queue gewartet hat, minimieren
- **Reaktionszeit** → Zeitspanne von der Einreihung in **ready**-Queue bis zur ersten Reaktion(wichtig bei Time-Sharing-Umgebung)

OPTIMIERUNGEN BEI SCHEDULING-ALGORITHMEN

- Maximale CPU-Auslastung
- Maximaler Durchsatz
- Minimale Durchlaufzeit — (Real-Time-Systeme!)
- Minimale Wartezeit
- Minimale Reaktionszeit — (Multi-Tasking-Systeme!)



- Je nach Wahl der Scheduling Algorithmen und Datenstrukturen lassen sich die einzelnen Eigenschaften optimieren. Es ist aber **nicht möglich** alle Eigenschaften zu optimieren!
- D.h. unterschiedliche System benötigen unterschiedliche Algorithmen

SCHEDULING ALGORITHMEN

FIRST COME FIRST SERVED (FCFS)

Prozess werden genau in der Reihenfolge abgearbeitet wie Sie eingetroffen sind

$$P_1 \rightarrow 24ms$$

$$P_2 \rightarrow 3ms$$

$$P_3 \rightarrow 3ms$$

- Angenommen die Prozesse werden in der Reihenfolge P_1, P_2, P_3 eingereicht



- Wartezeit: $P_1 = 0ms, P_2 = 24ms, P_3 = 27ms$
- Durchschnittliche Wartezeit: $\frac{0ms+24ms+27ms}{3} = 17ms$

FIRST COME FIRST SERVED (FCFS)

- Angenommen die Prozesse werden in der Reihenfolge P_2, P_3, P_1 eingereicht



- Wartezeit: $P_1 = 6ms, P_2 = 0ms, P_3 = 3ms$
- Durchschnittliche Wartezeit: $\frac{6ms+0ms+3ms}{3} = 3ms$
- Dieser Wert ist wesentlich besser als im Fall 1 ($17ms$)
 - Entsteht aber mehr zufällig
- **Convoy-Effekt** → kurze Prozesse hinter langen Prozessen
 - Beispiel: 1 langer CPU-lastiger Prozess und viele IO-lastige Prozesse

FCFS ist non-preemptive

SHORTEST JOB FIRST (SJF)

- Die Idee bei *SJF* ist es die Prozesse so anzuordnen, dass **kürzere CPU-burst-lastige** Prozesse vor **längeren CPU-burst-lastigen** Prozessen bearbeitet werden
- Zu jedem Prozess muss also die (*zeitliche*) Länge des nächsten CPU-Bursts bekannt sein
 - Die Anordnung erfolgt dann anhand dieser Werte (*fast* $\ll$ *slow*)
- SJF ist optimal zum Erreichen der **niedrigsten durchschnittlichen Wartezeit** von Prozessen
 - Das Problem ist es aber zu wissen, wie lange der nächste CPU-burst sein wird

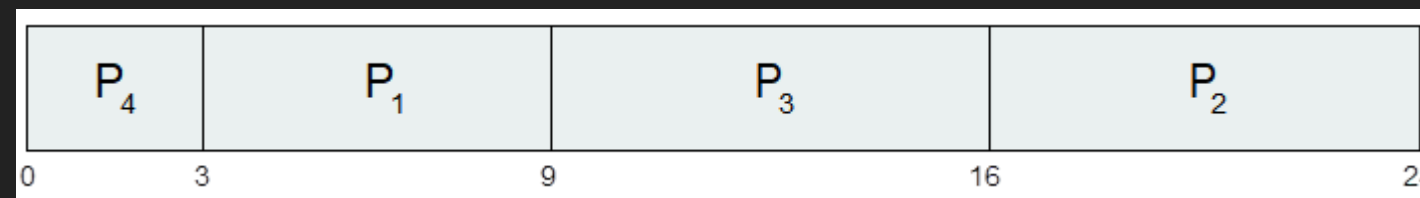
SHORTEST JOB FIRST (SJF): BEISPIEL

$$P_1 \rightarrow 6ms$$

$$P_2 \rightarrow 8ms$$

$$P_3 \rightarrow 7ms$$

$$P_4 \rightarrow 3ms$$



- Wartezeit: $P_4 = 0ms$, $P_2 = 3ms$, $P_3 = 9ms$, $P_4 = 16ms$
- Durchschnittliche Wartezeit: $\frac{0ms+3ms+9ms+16ms}{4} = 7ms$

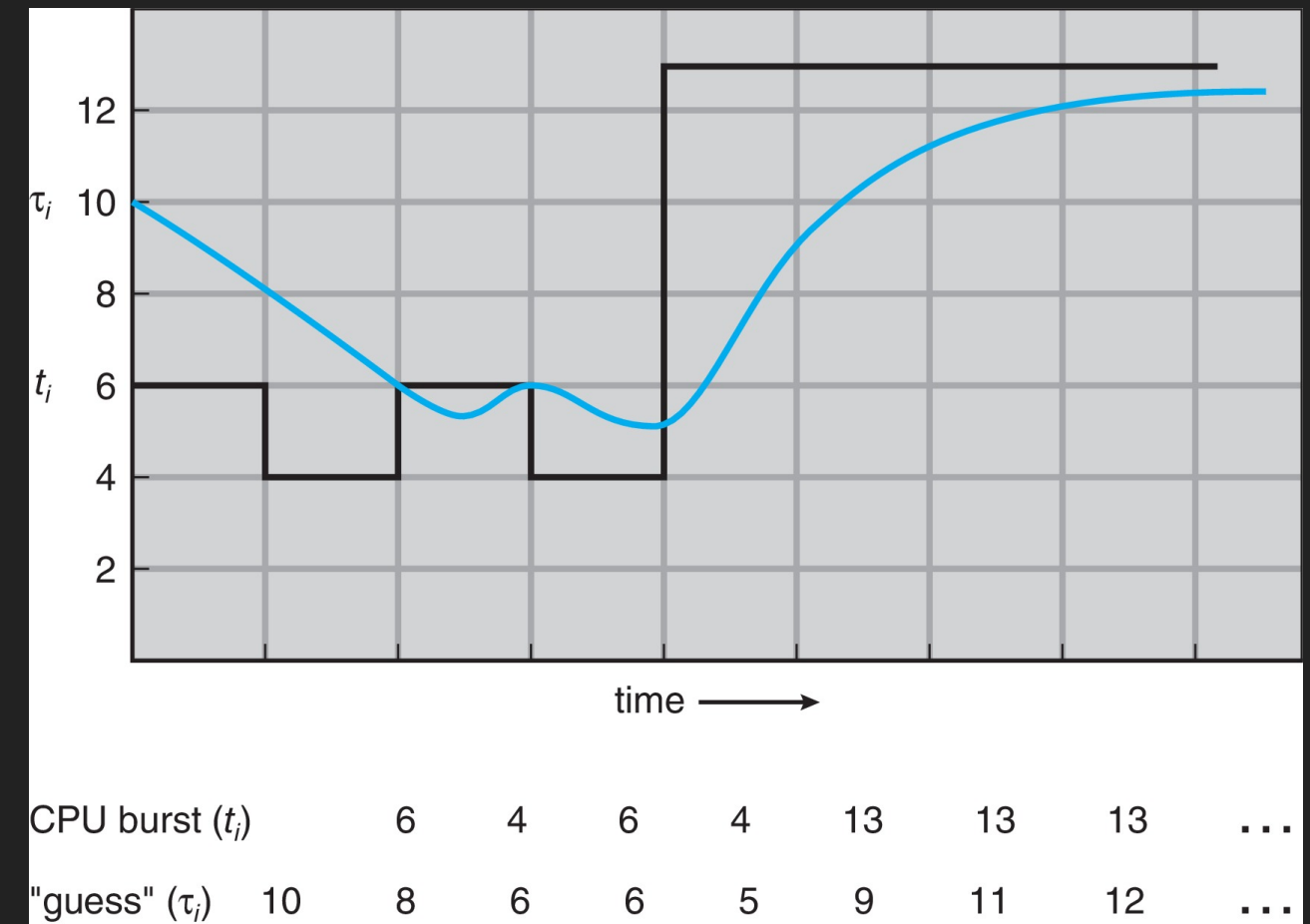
SHORTEST JOB FIRST (SJF): VORHERSAGE

- Da man nicht wissen kann, wie lange der nächste CPU-Burst sein wird, kann SJF auf der Basis von exponentieller Glättung (*exponential smoothing* oder *exponential average*) implementiert werden

$$\tau_{n+1} = \alpha * t_n + (1 - \alpha) * \tau_n$$

für α gilt: $0 \leq \alpha \leq 1$

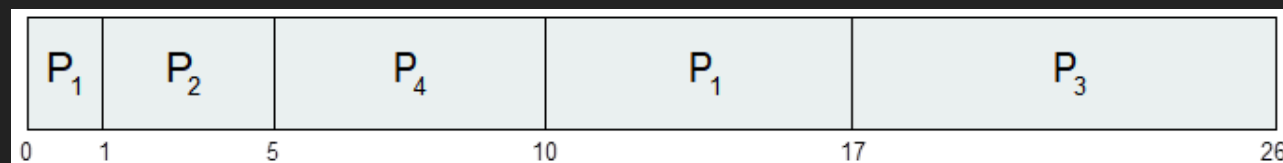
- Typischer Wert für $\alpha = \frac{1}{2}$
- Gleichverteilung von letztem CPU-Burst und vorherigen CPU-Bursts fließt gleichwertig in die Berechnung ein



SHORTEST JOB FIRST (SJF): PREEMPTIVE

- SJF kann non-preemptive oder preemptive implementiert sein
- Die preemptive-Variante heißt auch *Shortest Remaining Time First* (SRTF)

Prozess	Ankunftszeit	Burst-Zeit (berechnet)
P_1	0	$8ms$
P_2	1	$4ms$
P_3	2	$9ms$
P_4	3	$5ms$



- P_1 wird nach $1ms$ unterbrochen!
- Es folgen $P_2(4ms)$, $P_4(5ms)$
- dann wieder $P_1(7ms)$ und $P_3(9ms)$
- Durchschnittliche Wartezeit Preemptive (mit einbezogener Ankunftszeit):
 - $\frac{(10-1)+(1-1)+(17-2)+(5-3)}{4} = \frac{26}{4} = 6,5ms$
 - Reihenfolge: P_1, P_2, P_4, P_1, P_3
- Durchschnittliche Wartezeit Non-Preemptive:
 - $\frac{0+(8-1)(17-2)(12-3)}{4} = \frac{31}{4} = 7,75ms$
 - Reihenfolge: P_1, P_2, P_4, P_3

ROUND ROBIN (RR) PREEMPTIVE

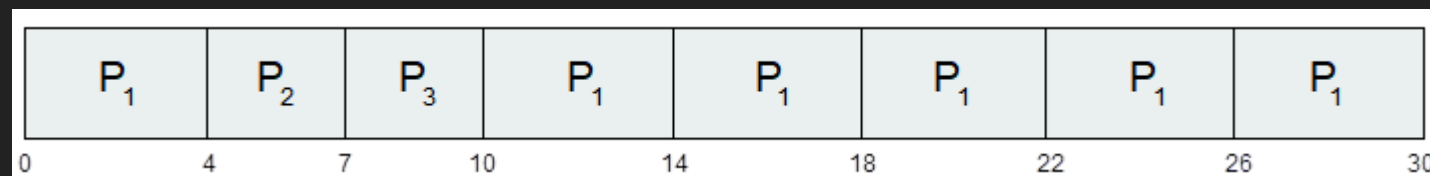
- Jeder Prozess bekommt eine bestimmte CPU-Zeit (*time quantum* q)
 - Typisch: 10-100 ms
- Nach Ablauf dieser Zeit wird der Prozess unterbrochen und an das Ende der **ready**-Queue platziert → Preemption
- Bei n Prozessen in der **ready**-Queue und *time quantum* q erhält jeder Prozess $\frac{1}{n}$ CPU-Zeit von maximal q ms auf einmal
 - Kein Prozess wartet länger als $((n - 1) * q)$ ms
- Timer löst alle q ms einen Interrupt aus und der nächste Prozess wird gescheduled
 - Bei q sehr groß → FIFO
 - Bei q sehr klein → zu viel Overhead wegen Kontextwechseln

ROUND ROBIN (RR) MIT 4 MS Q

$$P_1 \rightarrow 24ms$$

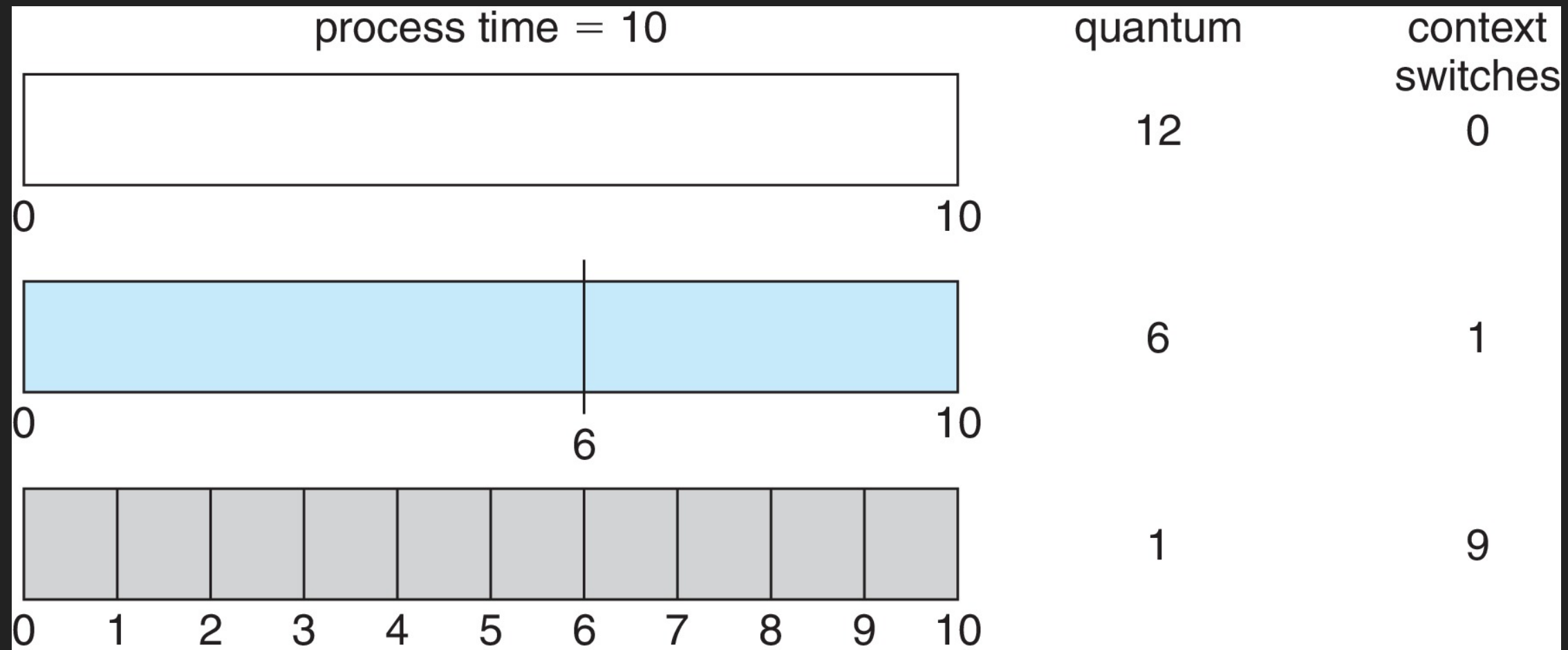
$$P_2 \rightarrow 3ms$$

$$P_3 \rightarrow 3ms$$



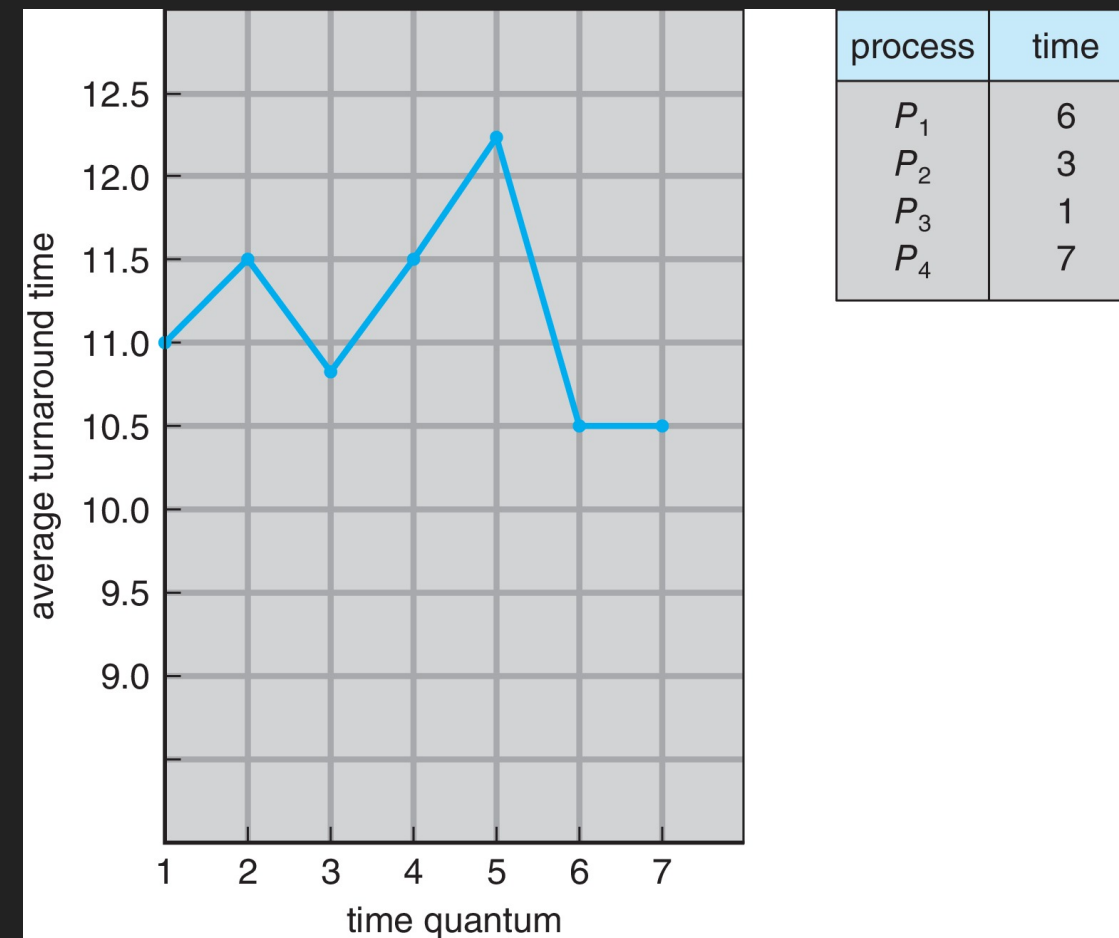
- Mittlere Durchlaufzeit höher als bei SJF
- Reaktionszeit niedriger als bei SJF

ROUND ROBIN (RR) QUANTUM ZEIT



- q ist typischerweise zwischen 10 ms und 100 ms bei context switch $< 10 \mu s$

ROUND ROBIN (RR) QUANTUM ZEIT



- Mittlere Durchlaufzeit variiert mit Quantum Zeit
- Richtwert: 80% der CPU-bursts sollten kürzer als q sein

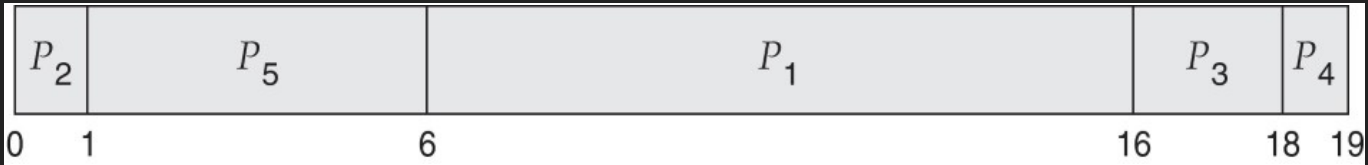
PRIORITY SCHEDULING

- Priority Scheduling führt einen Wert ein, mit dem ein **Priorität** einem Prozess zugeordnet werden kann
- Idee → Ordne die CPU dem Prozess mit der höchsten Priorität zu (z.B. kleinste Zahl = höchste Priorität)
 - Kann generell preemptive oder non-preemptive implementiert werden
- **SJF** war bereits eine Art von **Prioritätsscheduling**
 - Priorität invers zur Länge des nächsten (vorhergesagtem) CPU-Burst → je länger CPU-Burst umso niedriger die Priorität
- Problem → **Starvation** — Prozesse mit geringer Priorität werden ggf. niemals ausgeführt
- Lösung → **Aging** — Im Laufe der Zeit wird die Priorität eines Prozesses erhöht

PRIORITY SCHEDULING

Prozess	Burst-Zeit (berechnet)	Priorität
P_1	$10ms$	3
P_2	$1ms$	1
P_3	$2ms$	4
P_4	$1ms$	5
P_5	$5ms$	2

- Durchschnittliche Wartezeit: $8,2ms$



PRIORITY SCHEDULING + ROUND ROBIN

- Eine weitere Idee ist, Priority Scheduling mit Round Robin zu kombinieren
 - Primär Auswahl anhand der Priorität, bei gleicher Priorität

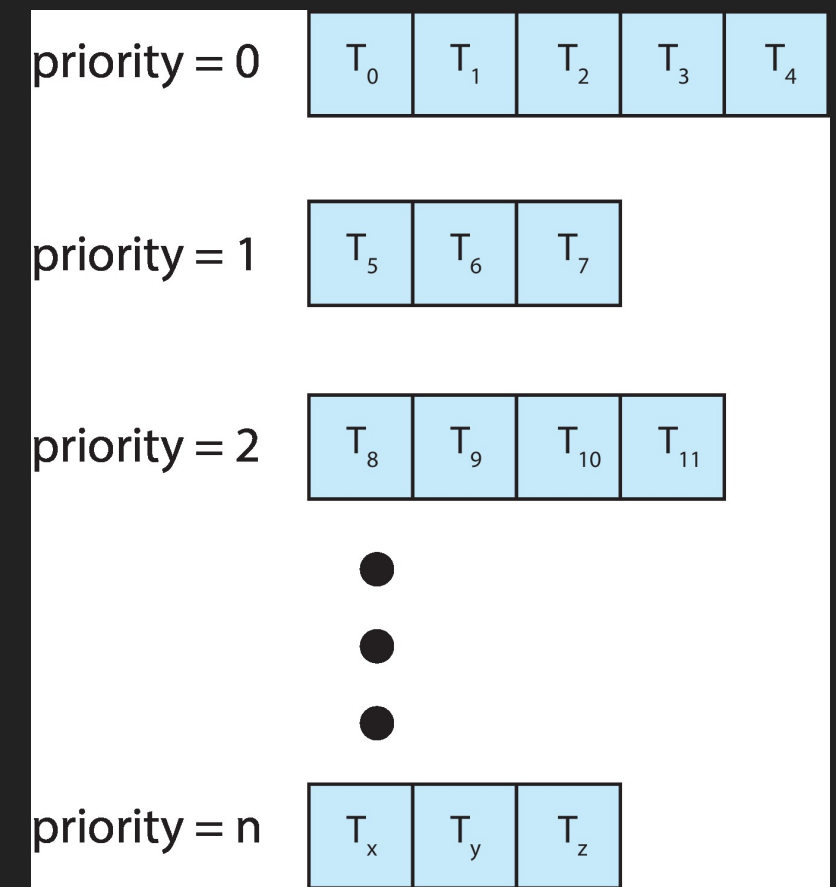
Prozess	Burst-Zeit (berechnet)	Priorität
P_1	$4ms$	3
P_2	$5ms$	2
P_3	$8ms$	2
P_4	$7ms$	1
P_5	$3ms$	3

- Annahme: $2ms$ Quantum



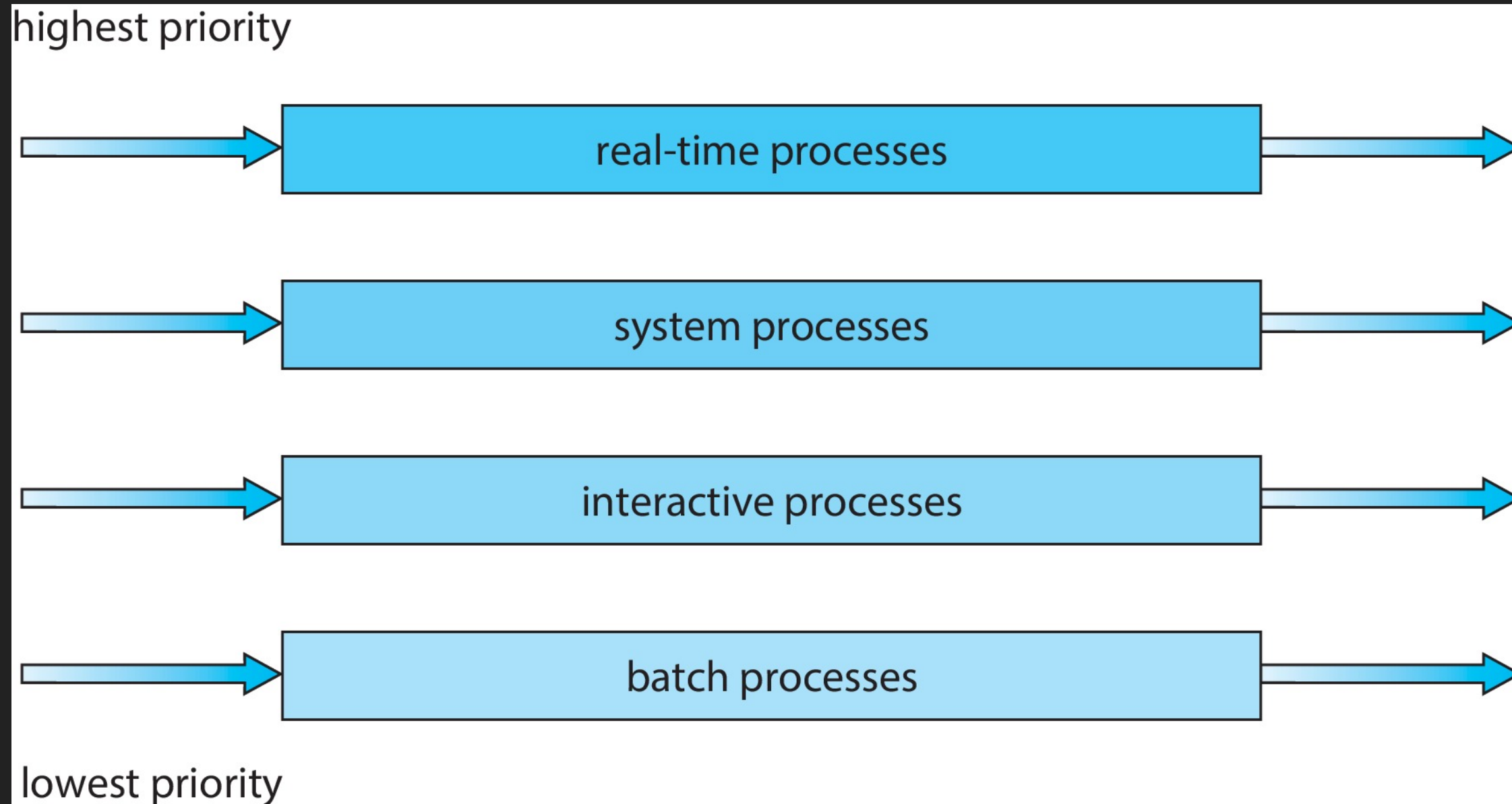
MULTILEVEL QUEUES

- Idee: Anstatt alles in einer einzelnen Scheduling-Queue zu verwalten wird für:
 - jeden Prioritätswert eine eigene Queue erzeugt
 - Queue-Gruppen mit definierten Prioritäts-Wertebereichen oder Typen erzeugt
 - Kann wiederum kombiniert werden
- Schedule den Prozess in der höchsten Prioritäts-Queue
 - Für jede Prioritäts-Queue kann der Auswahlalgorithmus und dessen Parameter bestimmt werden



MULTILEVEL QUEUES - GRUPPEN

- Klassifizierung anhand von Prozesstypen



MULTILEVEL FEEDBACK QUEUE

Idee: Ein Prozess kann zusätzlich zwischen Queues wechseln — So kann beispielsweise **Aging** implementiert werden

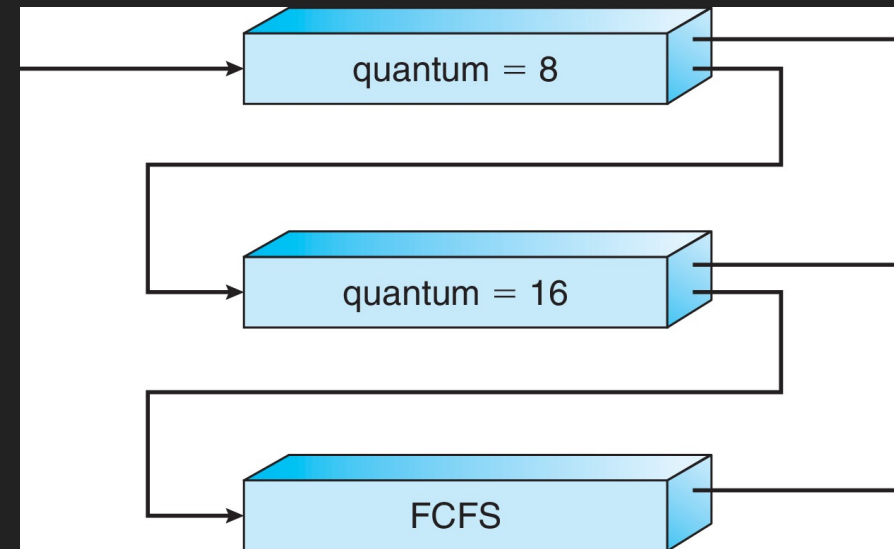
Ein **Multilevel Feedback Queue Scheduler** wird definiert anhand von Parametern:

- Anzahl an Queues
- Scheduling-Algorithmus für jede Queue
- Methoden die bestimmen wann ein Prozess **hochgestuft** oder **heruntergestuft** wird
- Methoden die bestimmen in welche Queue(-Gruppe) ein Prozess initial eingereiht wird

MULTILEVEL FEEDBACK QUEUE BEISPIEL

DREI QUEUES:

- Q_0 – RR mit Quantum $8ms$
- Q_1 – RR mit Quantum $16ms$
- Q_2 – FCFS



SCHEDULING:

- Ein neuer Task-CPU-Burst wird in Q_0 eingereiht (FCFS preempted $\rightarrow$ RR)
 - Bekommt maximal $8ms$, falls länger $\rightarrow$ verschieben in Q_1
- In Q_1 erhält er zusätzlich $16ms$ (wenn er dran kommt)
 - Reicht das immer noch nicht $\rightarrow$ verschieben in Q_2
- In Q_2 FCFS, aber keine Quantumvorgabe
 - Preemption nur falls Task mit höherer Priorität (Q_0 oder Q_1) vorhanden

THREAD SCHEDULING: ANMERKUNG

Wenn das Betriebssystem **Threads** als Konzept unterstützt, erfolgt das Scheduling auf Basis von **Threads** (... *und nicht auf Basis von Prozessen*)

Alle modernen Betriebssysteme unterstützen Threads in Form von **Single-Threaded Prozessen** und **Multi-Threaded Prozessen**

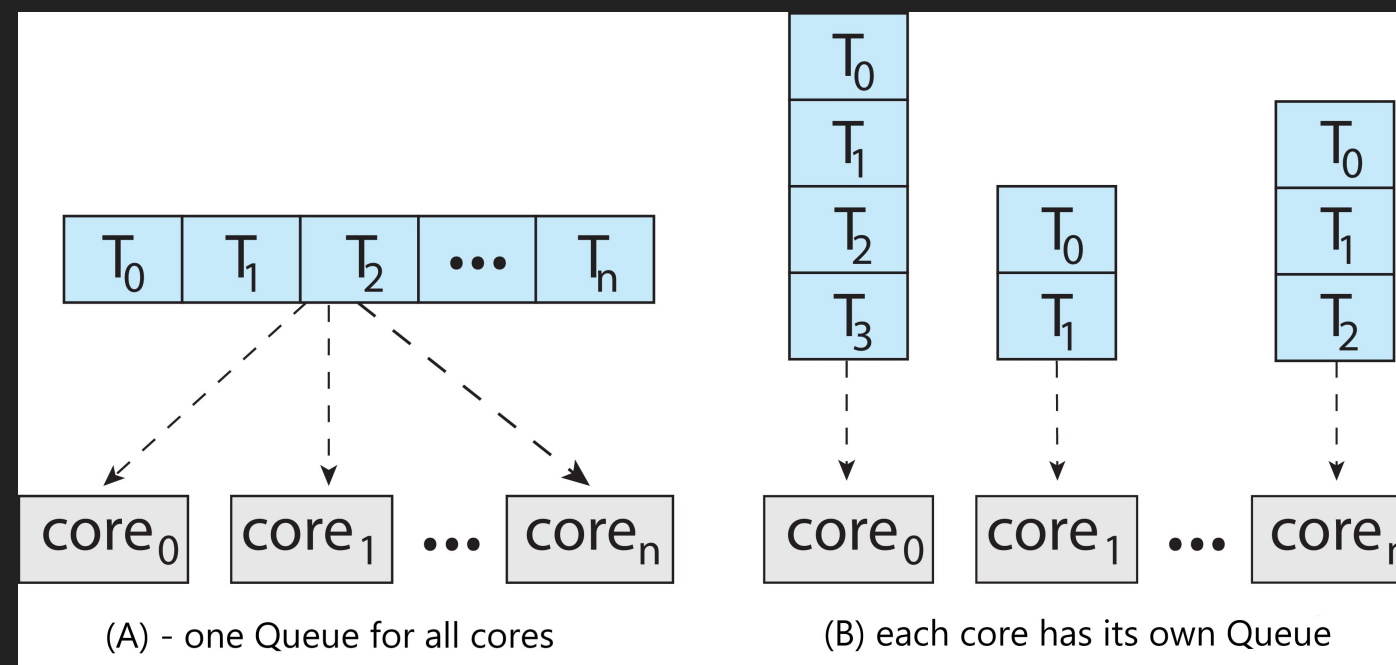


Implementierungen der beschriebenen Scheduling Konzepte verwenden heutzutage ausschließlich Threads!

MULTI PROZESSOR SCHEDULING

Die Scheduling Konzepte werden noch komplexer, sobald mehrere Prozessoren/Cores und Multi-Threading beteiligt sind

Bei symmetrischen Multi-Prozessor-Systemen (SMP) können **ready/run-Queues** in verschiedenen Varianten implementiert sein



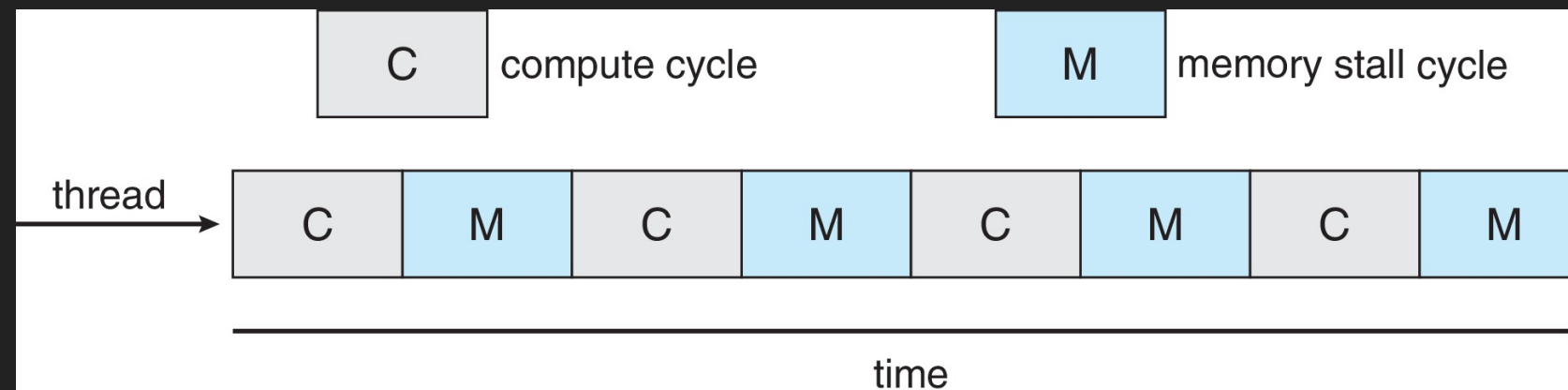
- Variante A: Alle Threads sind in einer einzelnen Queue
- Variante B: Für jeden Core wird eine separate Queue verwendet

MULTI PROZESSOR SCHEDULING

Typischerweise wird Option (B) präferiert, da ansonsten komplexe und vor allen **zeitlich intensive** Locking-Mechanismen zur Synchronisation implementiert werden müssen, da ein **massiv konkurrierender** Zugriff auf den geteilten Speicher der einzelnen Queue auftritt.

MULTICORE UND MULTITHREADING PROZESSOREN

- Heutige Prozessoren sind so schnell (viel schneller) als der Speicher, das ein sog. **memory stall**-Effekt auftritt
- Forschung hat gezeigt das die Cores eine signifikante Zeitspanne auf Daten aus dem Speicher warten
 - Insbesondere wenn Daten noch nicht im sehr viel schnelleren Prozessor Cache sind, tritt dies auf (cache-miss)

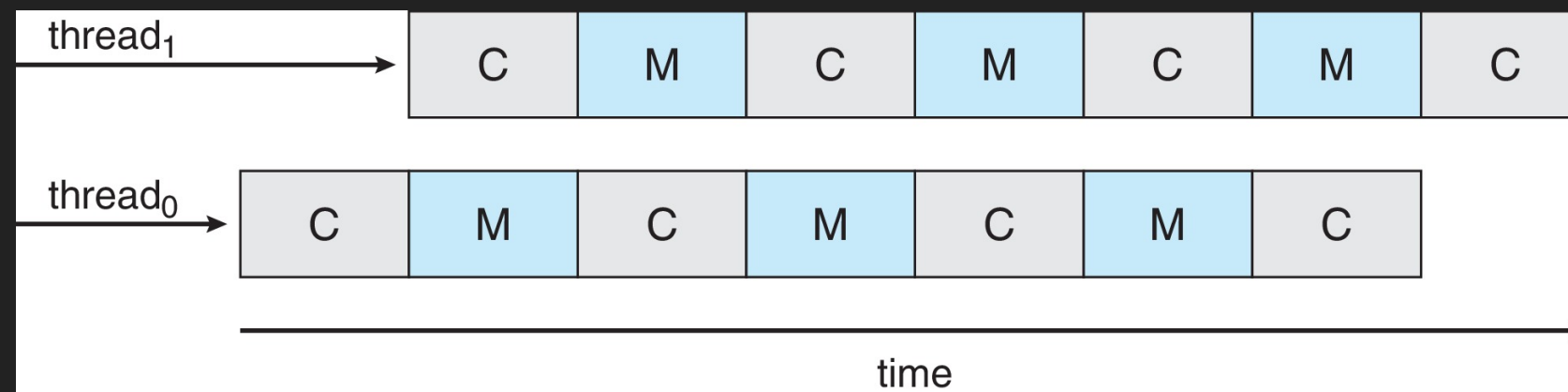


MULTICORE UND MULTITHREADING PROZESSOREN

Frage: Haben Sie eine Idee wie man diesen Umstand beheben bzw. verbessern könnte?

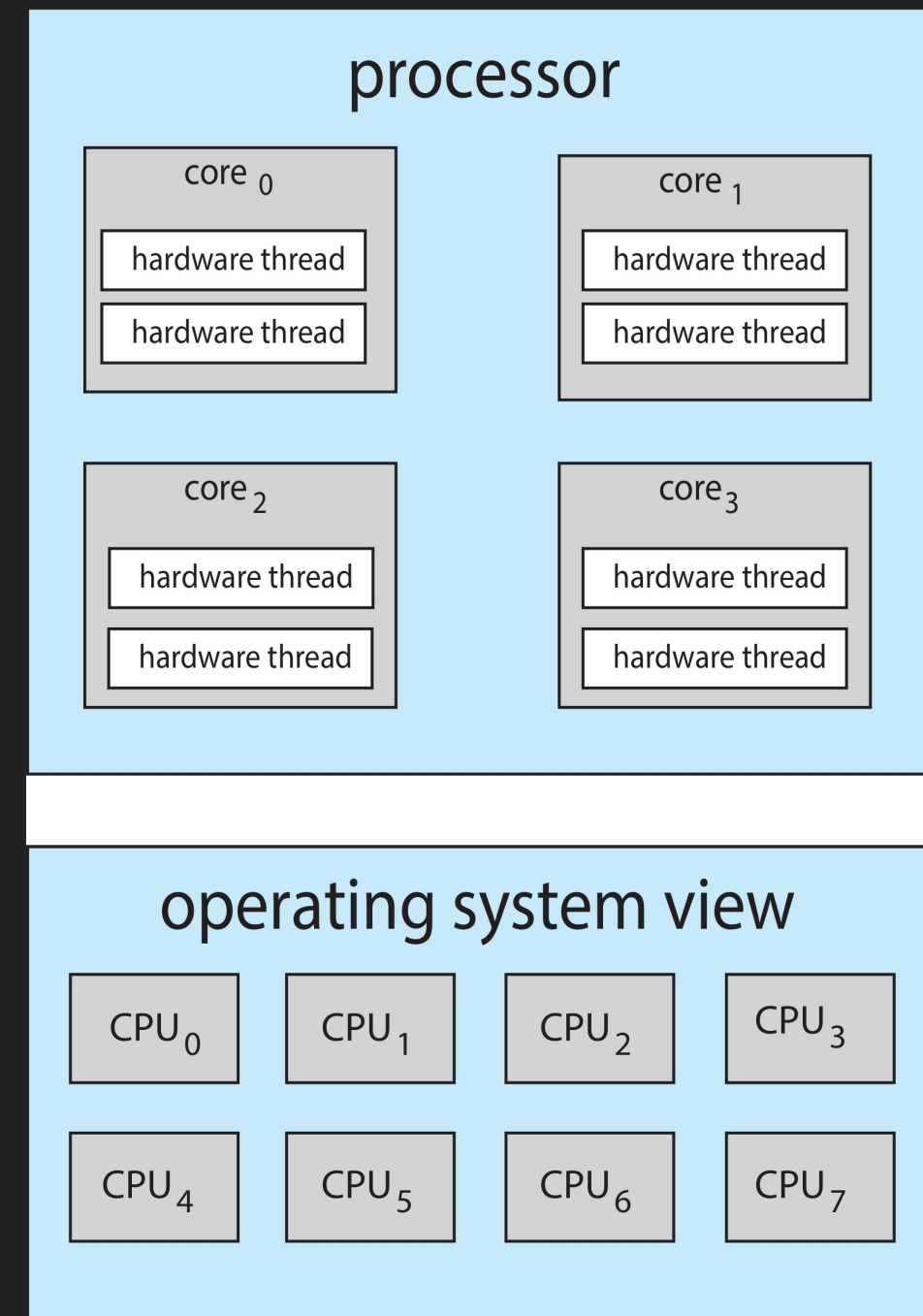
MULTICORE UND MULTITHREADING PROZESSOREN

- Lösung: Jeder Core hat effektiv mehr als einen (> 1) Hardware Thread
 - Tritt ein **memory stall**-Effekt bei einem Thread auf, wird zu dem anderen Thread gewechselt



MULTICORE UND MULTITHREADING SYSTEME

- **Chip-multithreading (CMT)** weist jedem Core mehrere Hardware Threads zu (Intel® nennt dieses Verfahren *Hyperthreading*)
- Ein Quad-Core System mit 2 Hardware Threads per Core wird im Betriebssystem als 8 logische Prozessoren gesehen



PROZESSOR/CORE AFFINITÄT

- Wenn ein Thread auf einem Prozessor/Core ausgeführt wurde, enthält der Cache des Cores Speicherinhalte die vom Thread verwendet wurden
- Dieses Verhalten bezeichnet man als Prozessor Affinität (*Processor Affinity*)
- Problem: *Load balancing* — gleichmäßige Verteilung von Threads auf unterschiedliche Prozessoren/Cores
 - Bei Wechsel des Cores, verliert man die Speicherinhalte des Caches!
- **Soft Affinity** → Das Betriebssystem versucht einen Thread (möglichst) auf dem gleichen Core auszuführen
 - Aber ohne Garantie
- **Hard Affinity** → Erlaubt es das ein Thread festlegt, auf welchen Cores er ausgeführt wird

REAL TIME SYSTEME

- **Soft real-time Systeme** — Es wird ein Prioritäts-basierter Scheduling Algorithmus verwendet und Real-time Threads/Tasks erhalten die höchste Priorität → Beispiel: Heizung mit Raumtemperaturüberwachung
 - Wann ein tatsächliches Scheduling erfolgt wird aber nicht vom Betriebssystem garantiert
- **Hard real-time Systeme** — Bestimmte Threads/Tasks müssen (garantiert) vollständig bearbeitet sein vor dem Ablauf einer **Deadline** → *Beispiel: Airbag*
 - Überschreitung der Deadline wird hierbei als vollständiges Versagen des Systems gewertet
- **Firm real-time Systeme** — siehe **Hard real-time** → Beispiel: Settop Box Video Frame Decoding — Jitter Effekte auf Fernseher
 - Überschreitung der Deadline wird hierbei **nicht** als vollständiges Versagen des Systems gewertet
 - Das Ergebnis/Resultat ist aber wertlos bzw. die Servicequalität wird schlechter

REAL TIME SYSTEME

- Alle modernen (multi-purpose) Betriebssysteme verwenden Prioritäts-basierte Scheduling Algorithmen und erfüllen somit implizit die Anforderung an **soft real-time** Systeme
- Bei **Firm** oder **Hard real-time** Systemen müssen spezielle Scheduling Algorithmen verwendet werden
 - Z.b. *Rate Monotonic Scheduling* für periodische Tasks
 - *Earliest Deadline First Scheduling* bei asymmetrisch auftretenden Ereignissen
- Bei beiden Verfahren muss bekannt sein wie lange ein CPU-Burst dauert und wann die Deadline abläuft
- Weiterhin muss das Betriebssystem speziell optimiert sein:
 - Minimale und eingehaltene Zeiten für Kontextwechsel, Interrupt Serviceroutinen, ...

SCHEDULING IN LINUX

HISTORY

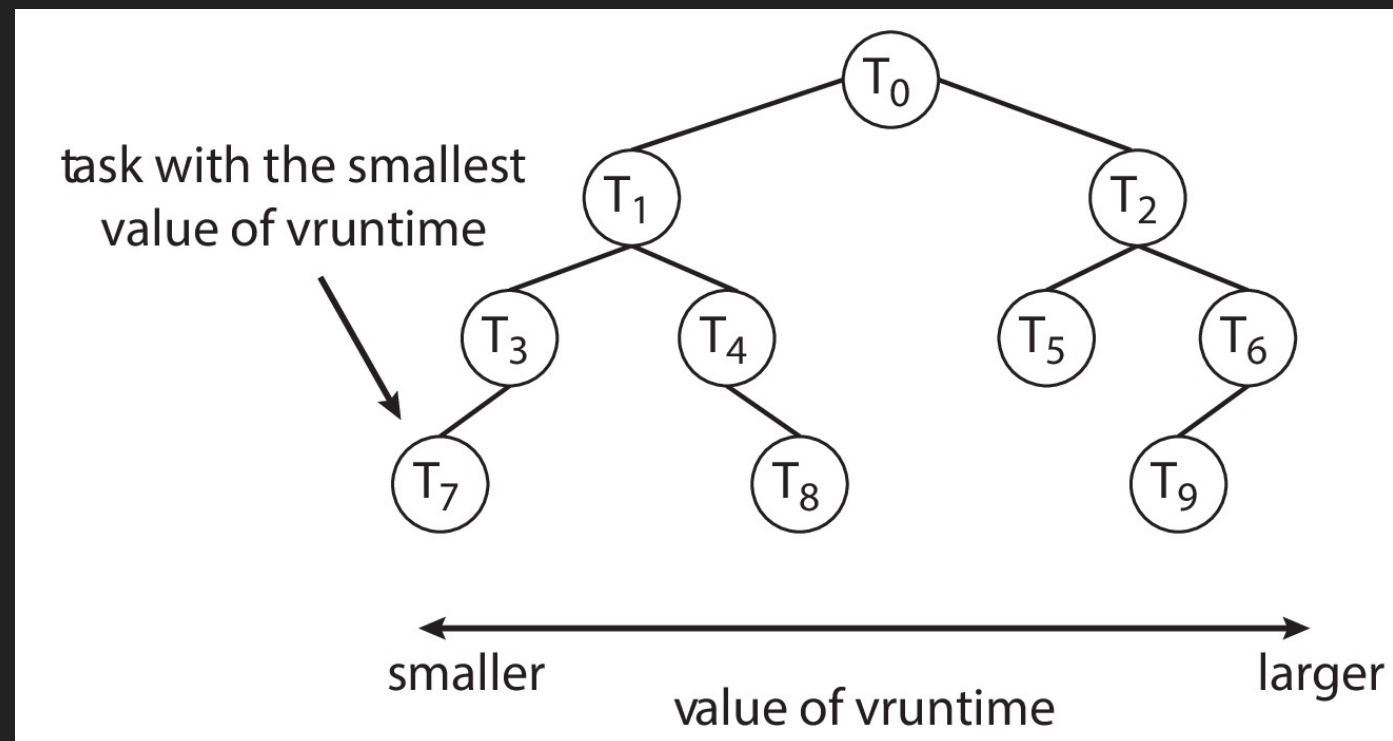
- Vor Kernelversion 2.5 → Variation an Standard Unix Scheduling Algorithmen
- Problem: War nicht auf SMP, sondern single-core Prozessoren ausgelegt — Daher schlechte SMP Performance
- 2.5 verwendete den $O(1)$ -Scheduler
 - Preemptive und Prioritäten-basiert
- Grundidee: 2 Prioritäten-Arrays per CPU (active und expired)
 - Wenn ein Task eine vorgegebene Zeit verbraucht hat: active → expired
 - Wenn active array leer → Austausch von active und expired
- Funktionierte generell gut für Server, aber schlecht für Desktops (wegen Interaktivität bei Multitasking Betrieb)

CURRENT: COMPLETELY FAIR SCHEDULER (CFS)

- Seit Kernelversion 2.6.3 wird der **Completely Fair Scheduler** (CFS) eingesetzt
- Verwendet sog. Scheduling Klassen (default P=[100 ... 139] und real-time P=[0 ... 99]) mit unterschiedlichen Grundprioritäten
 - Der Scheduler wählt jeweils den Task mit der höchsten Priorität in der höchsten Prioritätsklasse
- Quantum wird **dynamisch** berechnet und nennt sich *target latency*
 - Definiert einen Zeitintervall in der jedes **runnable**-Task mindestens einmal ausgeführt werden soll

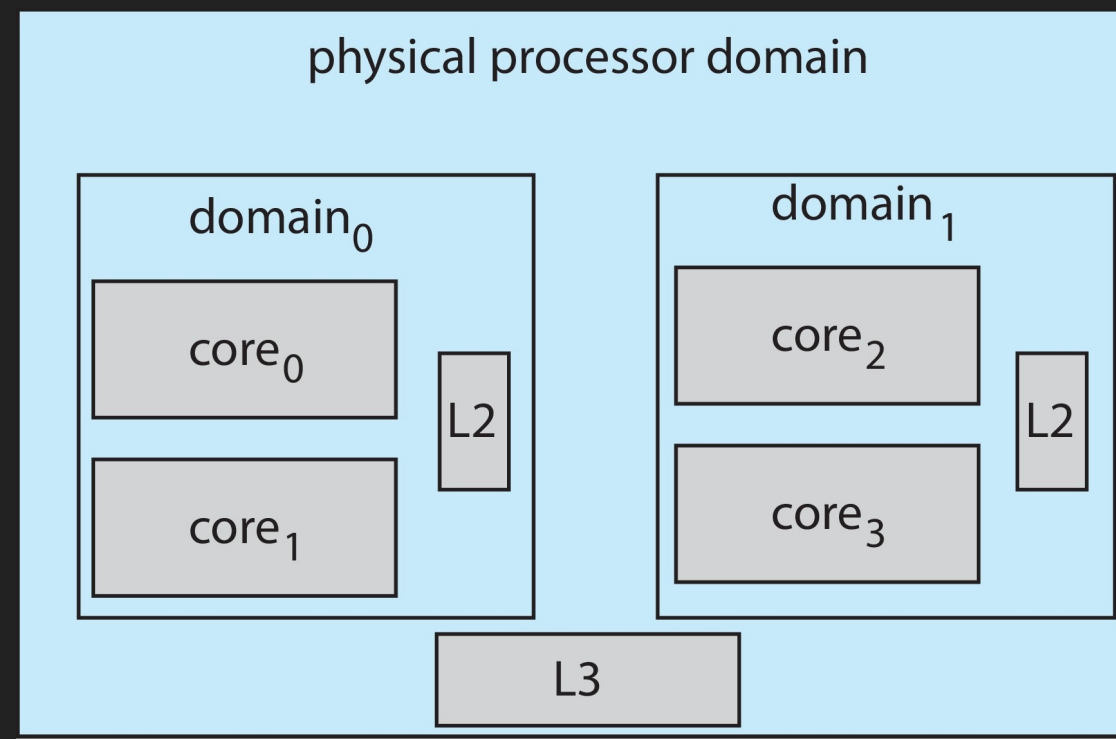
CURRENT: COMPLETELY FAIR SCHEDULER (CFS)

- Für jeden Task wird eine sog. Virtual Runtime (*vruntime*) berechnet
 - Hohe Priorität hat einen Faktor < 1 und niedrige Priorität > 1
 - Bevorzugt I/O intensive Tasks bei gleicher Priorität, da *vruntime* hier geringer
- Verwaltung in einer Baumstruktur (Red-Black-Tree)



CURRENT: COMPLETELY FAIR SCHEDULER (CFS)

- CFS unterstützt load-balancing (Verteilung von Tasks auf Cores bei SMP)
- CFS unterstützt und beachtet Prozessoraffinität



- Bevorzugt den Verbleib einer Task in einer Prozessordomäne
- Nur in Ausnahmefällen Migration in andere Domäne

NACHBEARBEITUNG: AUFGABEN UND FRAGEN

FRAGE/AUFGABE 1

Erklären Sie den Unterschied zwischen Preemptive und Non-Preemptive Scheduling Verfahren.

FRAGE/AUFGABE 2

Gegeben seien die folgenden Prozesse, deren Burst Time und Priorität

Prozess	Burst Time	Priorität
P_1	2	2
P_2	1	1
P_3	8	4
P_4	4	2
P_5	5	3

1. Zeichnen Sie die Gantt-Diagramme für die Scheduling Algorithmen: **FCFS**, Nicht-Preemptives Priority Scheduling (höhere Zahl = höhere Priorität)
2. Berechnen Sie für beide Varianten die Wartezeit pro Prozess und die durchschnittliche Wartezeit für einen kompletten Scheduling-Durchlauf

FRAGE/AUFGABE 3

Die Prozesse in der nachfolgenden Tabelle werden unter Verwendung eines **präemptiven Round-Robin**-Scheduling-Algorithmus zugeteilt. **Priorität**: numerisch höherer Wert = höhere Priorität! Die Länge eines *Zeitquantums* beträgt 5 Einheiten. Ein Prozess, mit einer höheren Priorität, verdrängt Prozesse mit niedriger Priorität **sofort**. Wenn ein Prozess durch eine höhere Priorität priorisiert ausgeführt wird, wird der verdrängte Prozess an **das Ende der Round Robin Warteschlange** gestellt.

Prozess	Priorität	Burst	Ankunftszeit
P 1	40	12	0
P 2	40	8	5
P 3	50	9	15
P 4	45	10	20
P 5	80	5	35
P 6	5	5	40

FRAGE/AUFGABE 4

Eine Variation des Round-Robin-Schedulers ist ein sog. regressiver Round-Robin-Scheduler. Dieser Scheduler weist jedem Prozess ein Zeitquantum und eine Priorität zu. Der Anfangswert des Zeitquantums beträgt 50 Millisekunden. Jedes mal wenn ein Prozess der CPU zugeordnet wurde und sein gesamtes Zeitquantum vollständig ausgenutzt hat (blockierte nicht für I/O), werden 10 Millisekunden zu seinem Zeitquantum hinzugefügt und seine Prioritätsstufe erhöht. (Das Zeitquantum für einen Prozess kann auf maximal 100 Millisekunden erhöht werden.) Wenn ein Prozess blockiert, bevor er sein gesamtes Zeitquantum ausnutzt, wird sein Zeitquantum um 5 Millisekunden reduziert, aber seine Priorität bleibt gleich.

- Welcher Typ von Prozessen (CPU-lastig oder I/O-lastig) wird von diesem Algorithmus bevorzugt?
 - Begründen Sie Ihre Antwort!

FRAGE/AUFGABE 4

- Erläutern Sie wozu die von Intel bezeichnet Technologie *Hyperthreading* verwendet wird.
- Halten Sie die Technologie für sinnvoll?
 - Begründen Sie Ihre Meinung!